# API de autenticación y usuarios

Documento de referencia para consumir la API de registro, login y perfil. Describe únicamente lo que está implementado en `backend/nodered-flows/flow-auth.json`.

## Base URL

| Entorno | URL |
|---|---|
| Local (Node-RED en Docker) | `http://localhost:1880` |
| Producción | `https://nodered.seguimiento360.com` |

Todos los cuerpos son JSON y las respuestas también. En producción el tráfico debe ir por HTTPS.

## Endpoints

| Método | Ruta | Auth | Descripción |
|---|---|---|---|
| POST | `/api/auth/register` | No | Crea usuario y tenant |
| POST | `/api/auth/login` | No | Valida credenciales y emite JWT |
| GET | `/api/auth/profile` | Bearer JWT | Devuelve el perfil del usuario |
| POST | `/api/auth/logout` | — | **No implementado**; el cliente borra el token |
| GET/PUT/DELETE | `/api/users...` | — | **No implementado**; no hay CRUD de usuarios |

## 1. Registro

```bash
API="http://localhost:1880"

curl -sS -X POST "$API/api/auth/register" \
  -H "Content-Type: application/json" \
  -d '{
    "email": "prueba@example.com",
    "password": "PruebaSegura123",
    "full_name": "Usuario de Prueba",
    "phone": "+573000000000"
  }' | jq
```

Respuesta 201:

```
{
  "success": true,
  "user": {
    "id": 1,
    "email": "prueba@example.com",
    "full_name": "Usuario de Prueba",
```

```
    "role": "customer"
  },
  "tenant_id": 1
}
```

Reglas:

- `email` y `password` son obligatorios.
- `password` debe tener al menos 8 caracteres.
- El email se normaliza a minúsculas.
- El registro crea el usuario y su tenant inicial.
- El rol inicial es `customer`.

Errores:

| Código | Cuándo |
|---|---|
| 400 | Falta email/password, password corto o email inválido |
| 409 | El email ya está registrado |
| 500 | Error interno de base de datos |

## 2. Login

```
TOKEN=$(curl -sS -X POST "$API/api/auth/login" \
  -H "Content-Type: application/json" \
  -d '{
    "email": "prueba@example.com",
    "password": "PruebaSegura123"
  }' | jq -r '.token')

echo "$TOKEN"
```

Respuesta `200`:

```
{
  "success": true,
  "token": "eyJhbGciOiJIUzI1NiIs...",
  "user": {
    "id": 1,
    "email": "prueba@example.com",
    "full_name": "Usuario de Prueba",
    "role": "customer",
    "tenant_id": 1
  }
}
```

El JWT contiene `user_id`, `email`, `role`, `tenant_id`, `iat` y `exp`. Expira a las 24 horas. Se firma con `jwtSecret` del `settings.js`.

Errores:

| Código | Cuándo |
|---|---|
| 400 | Falta email o password |

| Código | Cuándo |
|---|---|
| 401 | Credenciales inválidas |
| 403 | Cuenta desactivada |
| 500 | Error interno |

## 3. Perfil

```
curl -sS "$API/api/auth/profile" \
  -H "Authorization: Bearer $TOKEN" | jq
```

Respuesta 200:

```
{
  "success": true,
  "user": {
    "id": 1,
    "email": "prueba@example.com",
    "full_name": "Usuario de Prueba",
    "phone": "+573000000000",
    "role": "customer",
    "is_active": true,
    "created_at": "2026-10-22T15:00:00.000Z",
    "tenant_id": 1,
    "tenant_name": "prueba"
  }
}
```

Errores:

| Código | Cuándo |
|---|---|
| 401 | Sin token, token inválido o expirado |
| 404 | El usuario del token no existe |
| 500 | Error interno |

## 4. Logout

`POST /api/auth/logout` aparece en el manual general, pero **no está implementado** en `flow-auth.json`.

El logout real es del lado del cliente: la PWA elimina el token de `localStorage`. El token sigue siendo válido hasta su expiración de 24 horas. No hay lista de revocación.

Si se necesita invalidación inmediata, hay que implementar una lista de tokens revocados o rotar `JWT_SECRET` (lo que invalida todas las sesiones).

## 5. CRUD de usuarios

No existe. No hay endpoints para:

- Listar usuarios.
- Cambiar rol.
- Desactivar o eliminar usuarios.
- Resetear contraseñas.
- Gestionar usuarios por tenant.

El único endpoint administrativo es `POST /api/admin/generate-codes`, que genera códigos de vinculación de dispositivos, no usuarios.

Si el producto requiere administración de usuarios, debe diseñarse primero en la SPEC-02 y luego implementarse en Node-RED.

## 6. Secuencia completa de prueba

```
API="http://localhost:1880"
EMAIL="prueba-$(date +%s)@example.com"

# 1. Registro
curl -sS -X POST "$API/api/auth/register" \
  -H "Content-Type: application/json" \
  -d "{\"email\":\"$EMAIL\",\"password\":\"PruebaSegura123\",
\"full_name\":\"Prueba\"}" | jq

# 2. Login y captura del token
TOKEN=$(curl -sS -X POST "$API/api/auth/login" \
  -H "Content-Type: application/json" \
  -d "{\"email\":\"$EMAIL\",\"password\":\"PruebaSegura123\"}" | jq -r
'.token')

# 3. Perfil
curl -sS "$API/api/auth/profile" -H "Authorization: Bearer $TOKEN" | jq

# 4. Caso negativo: sin token
curl -sS -o /dev/null -w "%{http_code}\n" "$API/api/auth/profile"

# 5. Caso negativo: password incorrecto
curl -sS -o /dev/null -w "%{http_code}\n" -X POST "$API/api/auth/login" \
  -H "Content-Type: application/json" \
  -d "{\"email\":\"$EMAIL\",\"password\":\"incorrecta\"}"
```

Resultados esperados: `201`, `200`, `200`, `401`, `401`.

## 7. Implementación

| Archivo | Contenido |
|---|---|
| `backend/nodered-flows/flow-auth.json` | Nodos HTTP y funciones |
| `backend/nodered-flows/autenticacion-funciones.js` | Código de registro, login y perfil |
| `docs/firmware` | No aplica; el firmware no consume estas rutas |

Dependencias dentro del contenedor Node-RED: `pg`, `bcryptjs`, `jsonwebtoken`.

## 8. Seguridad

- El registro está abierto; en producción conviene limitar tasa y añadir verificación de email.
- No hay bloqueo por intentos fallidos.
- No hay refresh token.
- No hay revocación de JWT.
- El token se guarda en `localStorage`; evaluar `HttpOnly` cookies si se migra el frontend.
- No se deben registrar contraseñas ni tokens completos en logs.